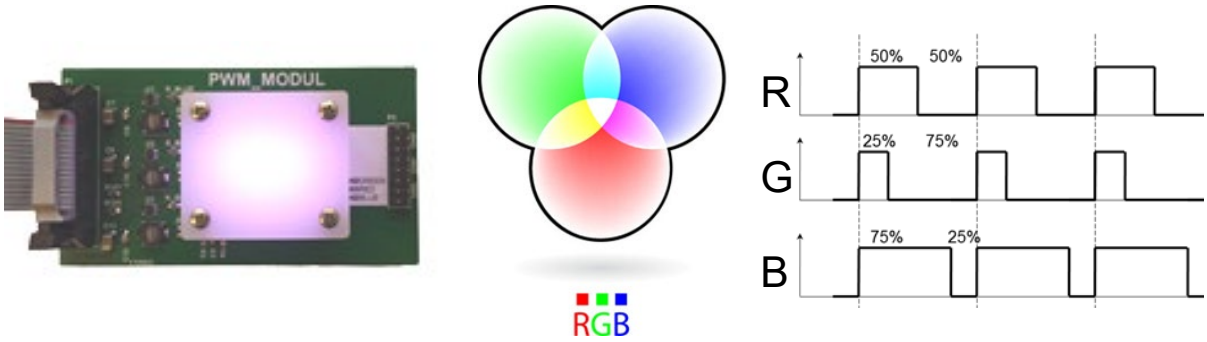
CT Praktikum: Timer und PWM

1 Einleitung

In diesem Praktikum setzen Sie die zwei 16-bit Timer TIM3 und TIM4 ein, um die Farben von dreifarbigen LEDs zu kontrollieren. TIM4 erzeugt im Sekundentakt einen Interrupt. TIM3 produziert drei synchrone PWM Signale um die drei LED Farben rot, grün und blau (RGB) anzusteuern. Der Reload Wert in TIM3 bestimmt die gemeinsame Periode der PWM Signale. Die Werte in den zugehörigen Compare Registern legen den individuellen Tastgrad (Duty Cycle) der einzelnen Farbsignale fest. Der Tastgrad wiederum bestimmt die Intensität mit welcher die entsprechende Farbe leuchtet. Durch Mischen der drei Grundfarben können alle anderen Farben erzeugt werden. Abbildung 1 zeigt das Prinzip.



source: colourbox

Abbildung 1: Ansteuern von RGB LEDs

Verwenden Sie für den Zugriff auf die LEDs und die Schiebeschalter des CT Boards die im File **`reg_ctboard.h`** definierten **`unions/structs`**.
Beispiele: **`CT_LED->BYTE.LED7_0`** und **`CT_DIPSW->BYTE.S7_0`**.

2 Lernziele

- Sie können mit einem Timer auf dem CT Board ein periodisches Interrupt erzeugen und eine zugehörige Interrupt Service Routine implementieren.
- Sie können mit einem Timer auf dem CT Board synchrone PWM Signale mit unterschiedlichen Duty Cycles erzeugen.
- Sie können die Funktion des Reload- und der Compare Register erklären.

3 Aufgaben

Verwenden Sie für die folgenden Aufgaben den zur Verfügung gestellten Projektrahmen. Dieser besteht aus einem Hauptprogramm **`main.c`** und einem Modul timer (**`timer.h`** und **`timer.c`**). Während das Hauptprogramm für die Ablaufsteuerung sorgt, stellt das Modul timer die Funktionen für die Zugriffe auf die verwendeten Timer zur Verfügung.

3.1 TIM4 – Interrupt im Sekundentakt

Der Timer TIM4 soll im Folgenden so initialisiert werden, dass er jede Sekunde einen Interrupt auslöst. In der zugehörigen Interrupt Service Routine **`TIM4_IRQHandler()`** sollen die LED31_24 im Sekundentakt blinken, d.h. alle LEDs ON, dann alle LEDs OFF.

Initialisierung TIM4

Ergänzen Sie die Funktion **`tim4_init()`**. Dabei müssen der Timer TIM4 konfiguriert und der zugehörige Interrupt freigeschaltet werden.

Hinweise

- Verwenden Sie für die Zugriffe auf die Kontrollregister des Timers, das in **`reg_stm32f4xx.h`** definierte **`struct reg_tim_t`** sowie das im gleichen File definierte Macro für die Basisadresse des Timers.
- Der Clockeingang **`CK_INT`** der verwendeten Timer wird bereits vor dem Start von **`main()`** auf 84 MHz gesetzt. Ebenso ist die Initialisierung einiger Timer Register am Anfang von **`tim4_init()`** bereits vorgegeben. Diese müssen nicht verändert werden.
- Wählen Sie den Prescaler Wert so, dass TIM4 in Zählschritten von 100 us (10 kHz) zählt.
- Setzen Sie den Wert des Reload Registers so, dass jede Sekunde ein Unterlauf erfolgt.
- Laden Sie für das Register CR1 den im Reference Manual beschrieben Defaultwert 0x0000. Wählen Sie zusätzlich die Richtung Downcounter.
- Enablen Sie im Timer TIM4 das Auslösen eines Interrupts bei Überlauf (Bit UIE in Register DIER). Disablen Sie die andern Interruptquellen und DMA Requests indem Sie alle anderen Bits auf null setzen.
- Enablen Sie ganz am Schluss den Timer TIM4 über das Bit CEN.
- Das Enablen des TIM4 Interrupts im NVIC am Ende der Funktion ist bereits vorgegeben (Register ISER0 – SETENA Bits).

Interrupt Service Routine (ISR)

Wenn der Timer TIM4 überläuft wird das UIF Bit gesetzt und die zugehörige Interrupt Service Routine **`TIM4_IRQHandler()`** aufgerufen. Ergänzen Sie den vorgegebenen Funktionsrahmen. In der ISR müssen das UIF Bit gelöscht werden und die LEDs angesteuert werden.

Hinweise

- Verwenden Sie für das Löschen des UIF Bits die vorgegebene Funktion **`tim4_reset_uif()`**.

Verifikation

Rufen Sie im Hauptprogramm **`main()`** die von Ihnen ergänzte Intialisierungsfunktion für den Timer TIM4 auf. Danach folgt eine endlose **`while(1)`** Schleife, welche auf den Interrupt wartet. Überprüfen Sie die korrekte Funktion Ihres Programmes auf dem CT Board.

3.2 TIM3 – Setzen des RGB Wertes

In diesem Schritt ergänzen wir das bestehende Programm um die drei synchronen PWM Signale für die Ansteuerung der LEDs auf dem Zusatzboard zu erzeugen. Die gemeinsame Periode der drei Signale soll auf 5 ms (200 Hz) eingestellt werden. Die Wahl des Periodenwertes ist ein 'Design Entscheid'. Sie basiert auf Eigenschaften des verwendeten LED Treiberbausteines und den Teilerverhältnissen im Microcontroller.

Von den Schiebeschaltern sollen die Duty Cycles der einzelnen Farben als 4-bit Werte eingelesen werden:

- S3..S0 Channel 1 P6.4 Rot
- S11..S8 Channel 2 P6.5 Grün
- S19..S16 Channel 3 P6.0 Blau

Die eingestellten Werte werden durch eine Multiplikation skaliert und in die entsprechenden Compare Register (CCR1 – CCR3) geladen. Bei einem Schalterwert 0x0 soll ein Duty Cycle von 0% entstehen, wogegen beim Schalterwert 0xF ein Duty Cycle von 100% entstehen soll. Der Bereich dazwischen soll in 15 gleich grosse Schritte eingeteilt werden. So entsteht beispielsweise bei der Schalterstellung 0x3 ein Duty Cycle von 3 / 15 ≙ 20 %.

Initialisierung TIM3

Hinweise

- Die Konfiguration der GPIO Pins ist bereits vorgegeben. Sie sind als PWM Ausgänge für den Timer TIM3 geschaltet.
- Ebenso ist die Initialisierung einiger Timerregister bereits vorgegeben.
- Konfigurieren Sie den Timer TIM3 als Upcounter. Alle anderen Bits im CR1 Register sollen auf 0 gesetzt werden.
- Wählen Sie entsprechende Werte für Prescaler und Reload damit bei möglichst grosser Ausnützung des Zählbereiches eine Frequenz von 200 Hz entsteht.
- Setzen Sie die Output Compare Modes der drei Channels auf Mode 1 (Bits OCxM im Register CCMR1 bzw. CCMR2)
- Enablen Sie die Outputs der drei Channels (Bits CCxE im CCER Register). Alle anderen Bits des Registers sollen auf 0 gesetzt werden.
- Enablen Sie den Timer TIM3 am Ende der Funktion.

Ergänzen Sie die Funktion **`tim3_init()`** mit der Konfiguration der Kontrollregister für den Timer TIM3.

Setzen der Duty Cycles

Lesen Sie im Hauptprogramm innerhalb der Endlosschleife die Werte der Schiebeschalter ein, skalieren Sie diese und setzen Sie die drei Duty Cycles mit der Funktion **`tim3_set_compare_register()`**. Diese ist als Rahmen vorgegeben und muss ergänzt werden.

Welcher Wert muss für einen Duty Cycle von 100% ins Compare Register geladen werden?

_____

Welcher Wert muss für einen Duty Cycle von 0% ins Compare Register geladen werden?

_____

Der Bereich dazwischen soll in 15 gleich grosse Schritte eingeteilt werden. Wie gross ist die Schrittweite und damit der Skalierungswert?

_____

Verwenden Sie diesen Wert für die Skalierung der eingelesenen 4-bit Werte.

Verifikation

Schliessen Sie das Zusatz Board an **Port P6 des CT Boards** an. Überprüfen Sie die korrekte Funktion Ihres Programmes. Lassen sich die Farben wie gewünscht einstellen?

Messen Sie die drei PWM Signale mit dem Logic Analyzer. Verwenden Sie für den Anschluss den vorbereiteten Stecker auf dem Zusatzprint. Siehe Abbildung 2. Halten Sie das Bild fest (zeigen bei der Abgabe).

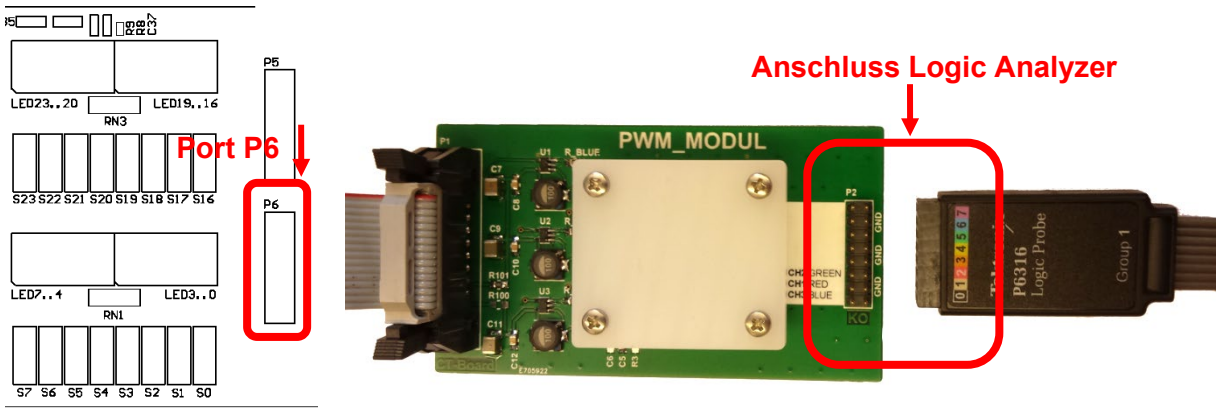


Abbildung 2: Messen der PWM Signale auf dem Zusatzprint

3.3 Periodisches Wechseln von Farben

Im dritten Schritt sollen die Farben zusätzlich im Sekundentakt wechseln. Über den Schiebeschalter S31 soll der Benutzer zwischen den Aufgaben 3.2 und 3.3 umschalten können.

Ergänzen Sie dazu die in Aufgabe 3.1 erstellte ISR um einen zyklischen 4-bit Zähler in der vordefinierten Variable **`cycle_counter_4bit`**. Bei jedem Interrupt wird der Zähler inkrementiert.

Verwenden Sie den Zählerwert im Hauptprogramm. Der Zählerwert soll direkt den Duty Cycle von Channel 1 bestimmen. Der Duty Cycle von Channel 2 soll dagegen durch (0xF minus Zählerwert) entsprechen. Dadurch zählt Channel 1 hoch während Channel 2 runterzählt. Channel 3 soll weiterhin durch den Wert an den Schiebeschaltern S19..S16 kontrolliert werden.

3.4 Bewertung

Die lauffähigen Programme müssen präsentiert werden. Die einzelnen Studierenden müssen die Lösungen und den Quellcode verstanden haben und erklären können.

Bewertungskriterien	**Gewichtung**
Der Timer 4 funktioniert wie gefordert.	1/2
Das Setzen des RGB Wertes funktioniert wie gefordert und das Bild des Zeitverlaufs auf dem Logic Analyzer wird präsentiert.	1/4
Periodisches Wechseln der Farben implementiert.	1/4